PROJECT HANDOVER REVIEW

Medbot
How the project works

A simple, evidence-based explanation of the current code, book data, multilingual flow, libraries, and known issues.

The books were not used to train a new AI model. Their text was extracted and stored in search indexes.

Live application

Uses only the English-only FAISS index. Non-English questions are translated to English before search.

Separate experiment

A combined English, Hindi, and Bengali index exists, but the Flask server does not use it.

466
English TIFF scans stored locally

697
English chunks used by the live search index

3,057
Chunks in the unused multilingual index

Repository snapshot reviewed on 29 September 2026. No project source file or database was changed.

01 - DIRECT ANSWERS

What the previous developer built

Was the English medical book trained?
No. EasyOCR converted page images into text. A pre-trained MiniLM model converted the text into numeric vectors. FAISS stores those vectors for similarity search. This is indexing, not model training.

Were multilingual books trained separately?
No. One experimental combined index contains English, Hindi, and Bengali text. There is not one index per language. The live application ignores this combined index.

Which knowledge does the live app use?
`chatbot.py` loads `indexeng.faiss` and `metadataeng.pkl`. That pair contains English book text only.

How can it answer in many languages?
It detects the input language, sends the question to Google Translate, searches the English book, asks a local Phi-3 model to write an English answer, then translates the final answer back.

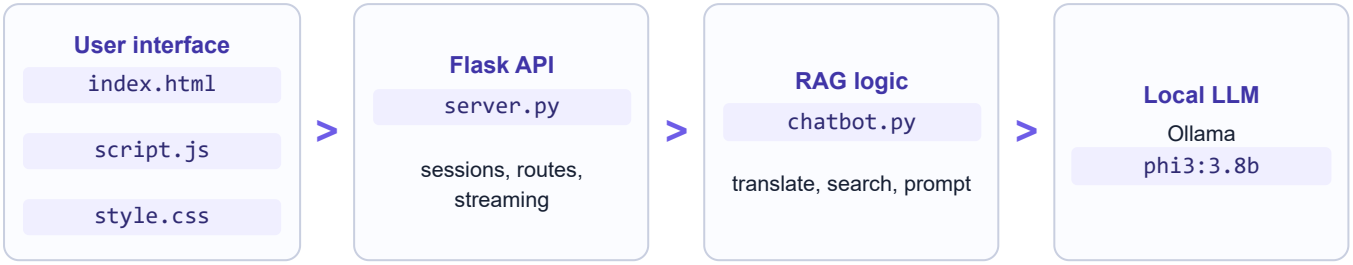
What was actually trained?
No trainable model in this repository was fine-tuned. Phi-3, MiniLM, EasyOCR, browser speech recognition, and Google Translate are all pre-built models or services. FAISS `IndexFlatL2` has no learning/training phase.

Important current bug

For a non-English question, the server first streams the English answer and then appends the complete translation. The user can see both answers joined together, and both are saved to chat history.

02 - ARCHITECTURE

Live application architecture



Knowledge path

1. English book scans in `content/.../WTIND`
2. OCR text in `metadataeng.pkl`
3. 384-number vectors in `indexeng.faiss`
4. Top matching chunks become the LLM context

Persistence path

1. Browser sends username, chat ID, and messages
2. `server.py` calls helper functions in `db.py`
3. SQLite stores users, chat IDs, and messages in `chat.db`

How the parts are started

- The frontend is expected to be served separately, usually from `http://127.0.0.1:5500`.
- The Flask API runs at `http://127.0.0.1:5000`.
- Ollama must already be running at `http://127.0.0.1:11434` with `phi3:3.8b` installed.
- The Flask root route returns only a status message; it does not serve the web interface.

03 - MULTILINGUAL BEHAVIOR

How multilingual input and output work

1. **Input:**
   The user types text, or the browser Web Speech API converts speech to text. The language dropdown affects speech recognition only.

2. **Detection:**
   `langdetect.detect()` guesses the language from the text.

3. **Translation:**
   If the language is not English, `GoogleTranslator` translates the question into English over the internet.

4. **Search:**
   English MiniLM creates a query vector. FAISS searches only the English index and returns up to five unique chunks.

5. **Generation:**
   The English chunks, the last six saved messages, and the translated question are sent to local Phi-3 through Ollama.

6. **Output:**
   English tokens are streamed to the browser. After generation finishes, Google Translate returns one complete translated answer.

What this design means

- Typed input can be any language that detection and Google Translate support.
- Voice choices in the UI are English, Hindi, Bengali, Tamil, Telugu, Gujarati, Urdu, and Assamese.
- The medical evidence is still English-only in the live path.

Failure behavior

- Translation requires internet access even though Hugging Face is forced into offline mode.
- Translation errors are silently ignored.
- If input translation fails, the code incorrectly treats the text as English.
- The English and translated answers are appended together.

Evidence: `chatbot.py` lines 32-46, 52-79, 85-103, 108-158, and 164-225; `script.js` lines 484-559.

English index versus multilingual index

Item	English-only pair	Combined multilingual pair
Status	USED LIVE	EXPERIMENTAL / UNUSED
Files	`indexeng.faiss` `metadataeng.pkl`	`index.faiss` `metadata.pkl`
Vector count	697	3,057
Vector size	384 numbers	384 numbers
Source coverage	452 unique English pages	1,539 unique pages across three languages
Language counts	English: 697 chunks	English: 973 chunks / 452 pages Hindi: 1,201 chunks / 477 pages Bengali: 883 chunks / 610 pages
Metadata fields	`text` , `image` , `chunk_id`	`text` , `source` , `language`
Embedding model in code	`all-MiniLM-L6-v2`	`paraphrase-multilingual-MiniLM-L12-v2`

English book

The repository stores 466 English TIFF pages. The live metadata references 452 of them as PNG names. The source route converts the matching TIFF file to PNG when the browser requests it.

Hindi and Bengali books

The combined metadata points to original Kaggle paths. Those Hindi and Bengali page files are not present in this repository, so their source images cannot be shown locally.

The notebook does not reproduce the repository

`pklfaissgenerate.ipynb` shows an English EasyOCR pipeline and reports 932 images, 2,342 chunks, and a 384-dimensional index. The checked-in artifacts contain 697 English chunks and 3,057 multilingual chunks. The notebook does not show how the multilingual files were produced and writes different filenames. A clean rebuild cannot be trusted from the current notebook alone.

05 - FILE GUIDE

What each code file does

File	Purpose	How it is implemented
`server.py`	Main Flask backend	Defines login, registration, chat management, history, source-image, logout, and streaming chat routes. Uses a signed Flask session cookie and sends Server-Sent Event style messages over a streamed POST response.
`chatbot.py`	Live retrieval and answer logic	Loads the English FAISS/metadata pair at import time, translates input to English, searches by L2 distance, builds a prompt with the last six messages, calls Ollama Phi-3, streams tokens, and translates the completed answer back.
`db.py`	SQLite data access	Opens one global connection and cursor, creates three tables, and provides functions for users, chat IDs, and messages.
`index.html`	Browser page structure	Contains the sidebar, login/register/logout buttons, chat area, input field, language dropdown, microphone button, logos, CSS link, and JavaScript link.
`script.js`	Browser behavior	Calls the Flask endpoints, renders chat lists and messages, reads the streaming response, shows source images, controls the sidebar, and uses the Web Speech API.
`style.css`	Visual design	Styles the layout, message bubbles, sidebar, buttons, typing animation, source images, responsive behavior, and microphone state. Several selector blocks are duplicated.
`chatbottest.py`	Older terminal test	Uses the English index, translation, and a non-streaming Ollama call with `llama3.2`. It is not imported by the server.
`chatbottest1.py`	Multilingual terminal experiment	Loads the combined multilingual index, prefers results matching the detected language, and asks Phi-3 to answer in that language. It is not imported by the server.
`help.py`	Developer debug helper	Prints all rows from the users table. The commented section tests Ollama. Running it can expose plaintext credentials.

06 - FILE GUIDE

Data, model-support, and generated files

Path	Role
`pklfaissgenerate.ipynb`	Google Colab build notebook. Mounts Google Drive, installs OCR/search libraries, converts TIFF pages to PNG, runs English EasyOCR, cleans and chunks text, creates MiniLM embeddings, and writes a FAISS index plus pickle metadata. Its recorded output does not match the checked-in index files.
`indexeng.faiss`	Live English vector index. It contains 697 vectors of dimension 384 in a flat L2 index.
`metadataeng.pkl`	Live English text and source mapping. Each result maps a vector position to OCR text, a PNG path, and a chunk ID.
`index.faiss`	Unused combined multilingual vector index with 3,057 vectors of dimension 384.
`metadata.pkl`	Unused multilingual text metadata with English, Hindi, and Bengali language labels and original Kaggle source paths.
`content/drive/MyDrive/WTIND/`	466 local English TIFF scans. These are the source pages displayed by the live UI after on-demand TIFF-to-PNG conversion.
`chat.db`	SQLite database containing users, chats, and messages. It is runtime data, not training data.
`images/full.png` `images/cdac.png`	Medbot and C-DAC logos used by the frontend.
`requirements.txt`	A large environment snapshot. It includes many packages not used by this application and omits several packages that are directly imported.
`venv/`	Ignored local virtual environment. It currently points to a Python installation that no longer exists, so its executables do not run.
`__pycache__/`	Generated Python bytecode cache. It is not source code.
`.gitignore`	Ignores `venv`, Python bytecode, and `.env`. It does not ignore the database, indexes, or generated data files.

07 - LIBRARIES

Which library does what

Library / service	Where used	Job
Flask	`server.py`	HTTP API, routes, JSON, sessions, and streaming responses.
Flask-CORS	`server.py`	Allows the frontend on port 5500 to send cookies to the API on port 5000.
SQLite / `sqlite3`	`db.py` , `help.py`	Stores accounts, chat IDs, and message history.
FAISS	chatbot files, notebook	Fast nearest-neighbor search over the stored embedding vectors.
sentence-transformers	chatbot files, notebook	Loads pre-trained MiniLM models and converts text into 384-number vectors.
NumPy	chatbot files, notebook	Creates the numeric arrays passed to FAISS.
EasyOCR	notebook	Reads English text from scanned page images. It is used only when building an index.
OpenCV	notebook	Reads TIFF files and writes PNG copies before OCR.
langdetect	chatbot files	Guesses the language code of the user's text.
deep-translator	`chatbot.py` , test	Calls Google Translate for input and output translation.
Requests	chatbot files	Sends HTTP requests to the local Ollama generation API.
Pillow	`server.py`	Converts a requested TIFF source page to PNG in memory.
Ollama + Phi-3	external local service	Generates the natural-language answer from the retrieved context. The model is not stored in this repository.
Web Speech API	`script.js`	Browser-provided speech-to-text. Availability and network behavior depend on the browser.
Font Awesome CDN	`index.html`	Downloads interface icons from the internet.

Dependency problem: `requirements.txt` does not list Flask, Flask-CORS, deep-translator, langdetect, EasyOCR, or OpenCV, even though the code/notebook imports them. It also contains many unused AI frameworks and providers.

Routes and database structure

Flask routes

Route	Purpose
`GET /`	API status text
`POST /register`	Create a user
`POST /login`	Set session user
`POST /logout`	Clear session
`POST /new_chat`	Create chat ID
`GET /get_chats`	List current user's chats
`POST /rename_chat`	Rename chat and messages
`POST /delete_chat`	Delete chat and messages
`GET /history/<id>`	Return saved messages
`POST /chat`	Stream answer and sources
`GET /source/<file>`	Serve or convert source page

SQLite tables

```
users
  username  TEXT PRIMARY KEY
  password  TEXT

chats
  chat_id   TEXT PRIMARY KEY
  username  TEXT

messages
  chat_id    TEXT
  role       TEXT
  message    TEXT
  created_at TIMESTAMP
```

What is missing

- No password hash.
- No numeric primary key for messages.
- No foreign keys between tables.
- No ownership filter in history, rename, or delete helpers.
- No explicit message ordering in `load_chat`.

Streaming format: The backend sends events with types `token`, `sources`, `done`, or `error`. The frontend manually splits the response on blank lines and updates one bot message as tokens arrive.

09 - ISSUES

Problems found, in priority order

Level	Issue	Why it matters
CRITICAL	**Passwords are plaintext**	Passwords are stored and compared as normal text. Registration/login payloads are also printed by the server, and `help.py` prints every user row.
CRITICAL	**Chat ownership is not enforced**	A logged-in user can submit another chat ID to rename or delete it. The history route does not even check login. Database helpers filter only by chat ID.
CRITICAL	**Unsafe medical behavior**	The prompt explicitly permits medicine dosages. There is no emergency triage, contraindication check, uncertainty handling, clinical review, or clear user warning. Retrieval rules do not guarantee the model will stay inside the book.
High	**Hardcoded secret and debug mode**	The session key is `super_secret_key` and Flask starts with `debug=True`. Sessions can be forged if exposed, and the debugger must not be used in production.
High	**Non-English answer is duplicated**	English tokens are already sent before the translated full answer is yielded. The UI and database receive English plus the translation joined together.
High	**Database concurrency is unsafe**	One global SQLite cursor is shared while Flask is started with `threaded=True`. Concurrent requests can interfere with each other.
High	**Translation failures are hidden**	Bare `except` blocks silently fall back. A failed Hindi/Bengali translation may be searched as if it were English, producing weak or misleading evidence.
High	**HTTP calls have no timeout/status check**	If Ollama or translation hangs or fails, a request can remain open. The Ollama stream skips malformed error lines instead of reporting a clear failure.

Portability and maintainability issues

Environment is not reproducible

- The checked-in virtual environment is broken because its base Python path no longer exists.
- Required direct packages are missing from `requirements.txt`.
- Ollama and the Phi-3 model are external prerequisites with no setup script.
- `HF_HUB_OFFLINE=1` requires MiniLM to already exist in the user's Hugging Face cache.

Index build is not reproducible

- The notebook output does not match either checked-in data pair.
- The multilingual index build process is missing.
- Only English source scans are present locally.
- The notebook can process duplicated TIFF and PNG versions of the same page.

Runtime assumptions

- Relative file paths require starting Flask from the project directory.
- CORS permits only one hardcoded frontend origin.
- The frontend depends on an internet CDN for icons.
- Registration reports success even when the username already exists because it uses `INSERT OR IGNORE`.

Code quality

- Unused import and duplicate `json` import in `server.py`.
- Many repeated CSS selector blocks override earlier rules.
- The experimental Phi-3 name contains leading spaces.
- No automated tests, README, configuration file, or deployment instructions.

Practical handover conclusion

The project is a local RAG prototype. It combines OCR book text, vector search, a local Ollama model, online translation, a Flask API, a plain JavaScript interface, and SQLite chat history. The English flow is the only one wired into the web application. The multilingual index is an unfinished alternative experiment.

Recommended order before further feature work

1. Fix password storage, session configuration, and per-user chat authorization.
2. Fix the translated streaming response and add explicit translation failure handling.
3. Create a clean environment file and a fully reproducible index-build script.
4. Decide whether production should use translated English retrieval or the combined multilingual index.
5. Add medical safety rules and test the full question-to-source-to-answer path.